Java OOP Assignment

Payment System — Main Focus: Abstraction

1. Problem Statement

You are required to design a simple Payment System using Java Object-Oriented Programming concepts. A customer can make payments using different methods such as Credit Card, Bank Account, or Mobile Wallet. Each payment method has its own way of processing a payment, while some information and behavior are common to all payments.

Main Goal: The main focus of this assignment is **Abstraction**. You must identify the common behavior and design an abstract parent class, while leaving payment-specific implementation to the child classes.

2. Required Classes

Abstract parent class: Payment

The Payment class should contain common payment information such as:

- paymentId
- amount

It should also contain a concrete method such as `displayPaymentDetails()` to display common payment information.

It must contain an abstract method such as `processPayment()`. Do not provide the actual payment-processing implementation in the abstract parent class.

Child classes:

- CreditCardPayment
- BankPayment
- MobileWalletPayment

Each child class must extend Payment and provide its own implementation of `processPayment()`.

3. Abstraction Requirements ■

- Payment must be declared as an abstract class.
- Payment must not be instantiated directly.
- processPayment() must be an abstract method.
- Each child payment class must implement processPayment() differently.
- The user of the class should only need to know that a payment can be processed; the internal processing details should remain inside the specific child class.

4. Encapsulation Requirements

Payment information should not be directly exposed. Use appropriate access modifiers for fields such as `paymentId` and `amount`. Provide appropriate methods for accessing or updating the data where required.

5. Inheritance Requirements

Demonstrate inheritance using the following relationship:

```
Payment
  ↑
  ■■■ CreditCardPayment
  ■■■ BankPayment
  ■■■ MobileWalletPayment
```

6. Polymorphism Requirements

In the main method, use a Payment reference to refer to different child objects. Call processPayment() on each reference and observe that the appropriate child implementation is executed.

Conceptually, your program should demonstrate:

```
Payment → CreditCardPayment
Payment → BankPayment
Payment → MobileWalletPayment
```

7. Main Method Requirements

- Create at least three payment objects: Credit Card, Bank, and Mobile Wallet.
- Display the details of each payment.
- Process each payment.
- Show different processing behavior for each payment type.
- Do not create an object directly from the Payment abstract class.

8. Expected Output

Your exact output may be different, but it should communicate information similar to:

```
Payment ID: 101
Amount: 5000
Processing Credit Card Payment...

------------------------

Payment ID: 102
Amount: 10000
Processing Bank Payment...

------------------------

Payment ID: 103
Amount: 2500
Processing Mobile Wallet Payment...
```

9. OOP Pillars Checklist

OOP Pillar	Where to demonstrate
Abstraction ■	Payment abstract class + processPayment()

Inheritance	Payment classes extend Payment
Polymorphism	Payment reference referring to different child objects
Encapsulation	Payment fields + controlled access

10. Bonus Challenge — Optional

After completing the basic assignment, add another payment type such as CashPayment.

- Can you add the new payment type without changing the existing child classes?
- Does your design make it easy to support future payment methods?
- Explain your answer briefly in comments or a short note.

11. Important Instructions

- Do not copy a complete solution from the internet.
- The assignment is intentionally designed so that you decide the exact fields, constructors, and implementation details.
- Focus especially on understanding why abstraction is useful and why processPayment() belongs as an abstract method.
- You may use meaningful sample data and your own output messages.

Interview Practice Question: Why is Payment an abstract class, and why should processPayment() be implemented differently by each child class?